

1

10장 자바의 이벤트 처리

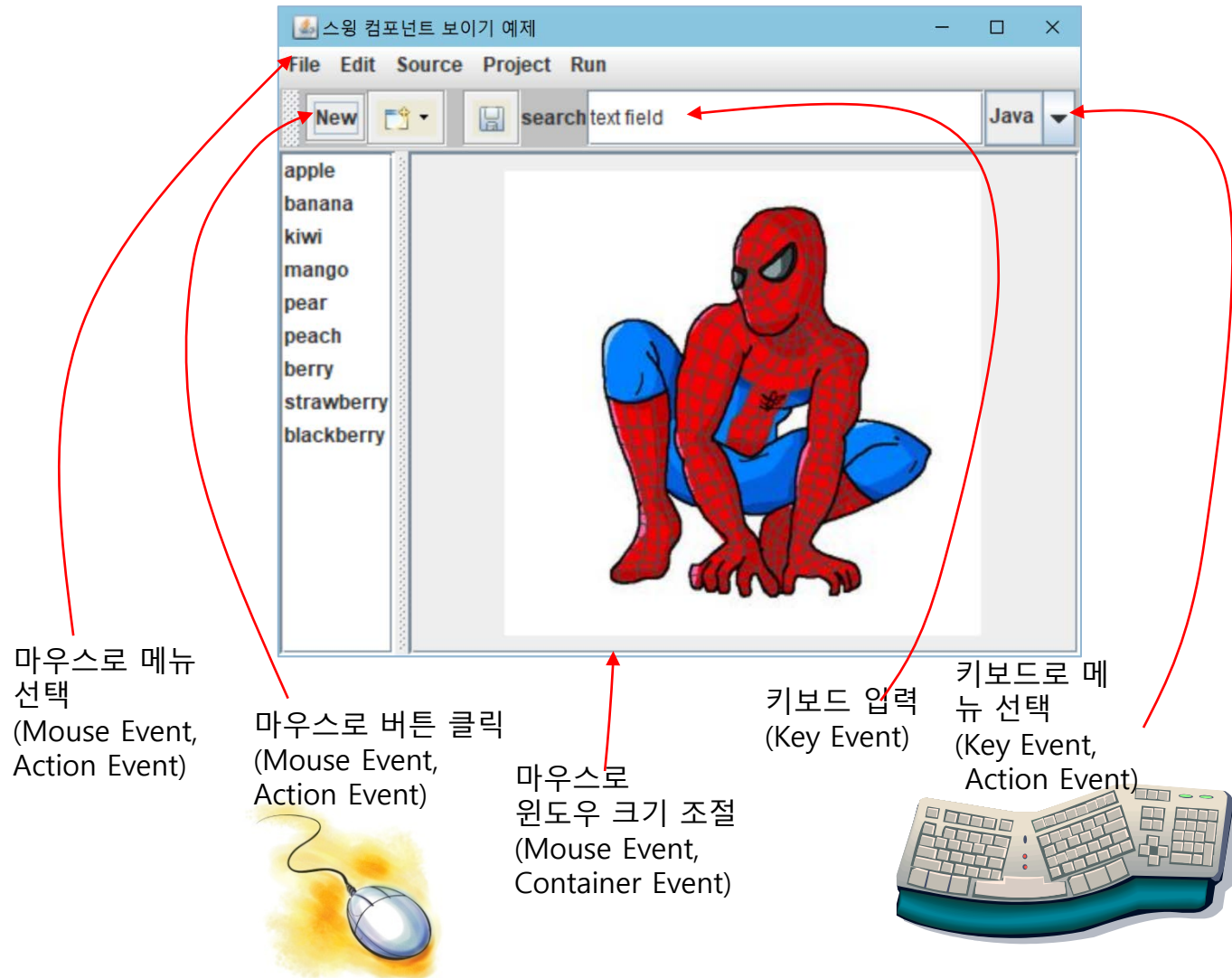
이벤트 기반 프로그래밍

2

- 이벤트 기반 프로그래밍(Event Driven Programming)
 - ▣ 이벤트 종류
 - 사용자의 입력 : 마우스 드래그, 마우스 클릭, 키보드 누름 등
 - 센서로부터의 입력, 네트워크로부터 데이터 송수신
 - 다른 응용프로그램이나 다른 스레드로부터의 메시지
 - ▣ 이벤트의 발생에 의해 프로그램 흐름이 결정되는 방식
 - 이벤트가 발생하면 이벤트를 처리하는 루틴(이벤트 리스너) 실행
 - 프로그램 내의 어떤 코드가 언제 실행될 지 이벤트 발생에 의해 전적으로 결정
 - ▣ 반대되는 개념 : 배치 실행(batch programming)
 - 프로그램의 개발자가 프로그램의 흐름을 결정하는 방식
- 이벤트 기반 프로그램의 구조
 - ▣ 이벤트 리스너 들의 집합
- 이벤트 처리 순서
 - ▣ 이벤트 발생(예 :마우스나 키보드의 움직임 혹은 입력)
 - ▣ 이벤트 객체 생성
 - 현재 발생한 이벤트에 대한 정보를 가진 객체
 - ▣ 이벤트 리스너 찾기
 - ▣ 이벤트 리스너 호출
 - 이벤트 객체가 리스너에 전달됨
 - ▣ 이벤트 리스너 실행

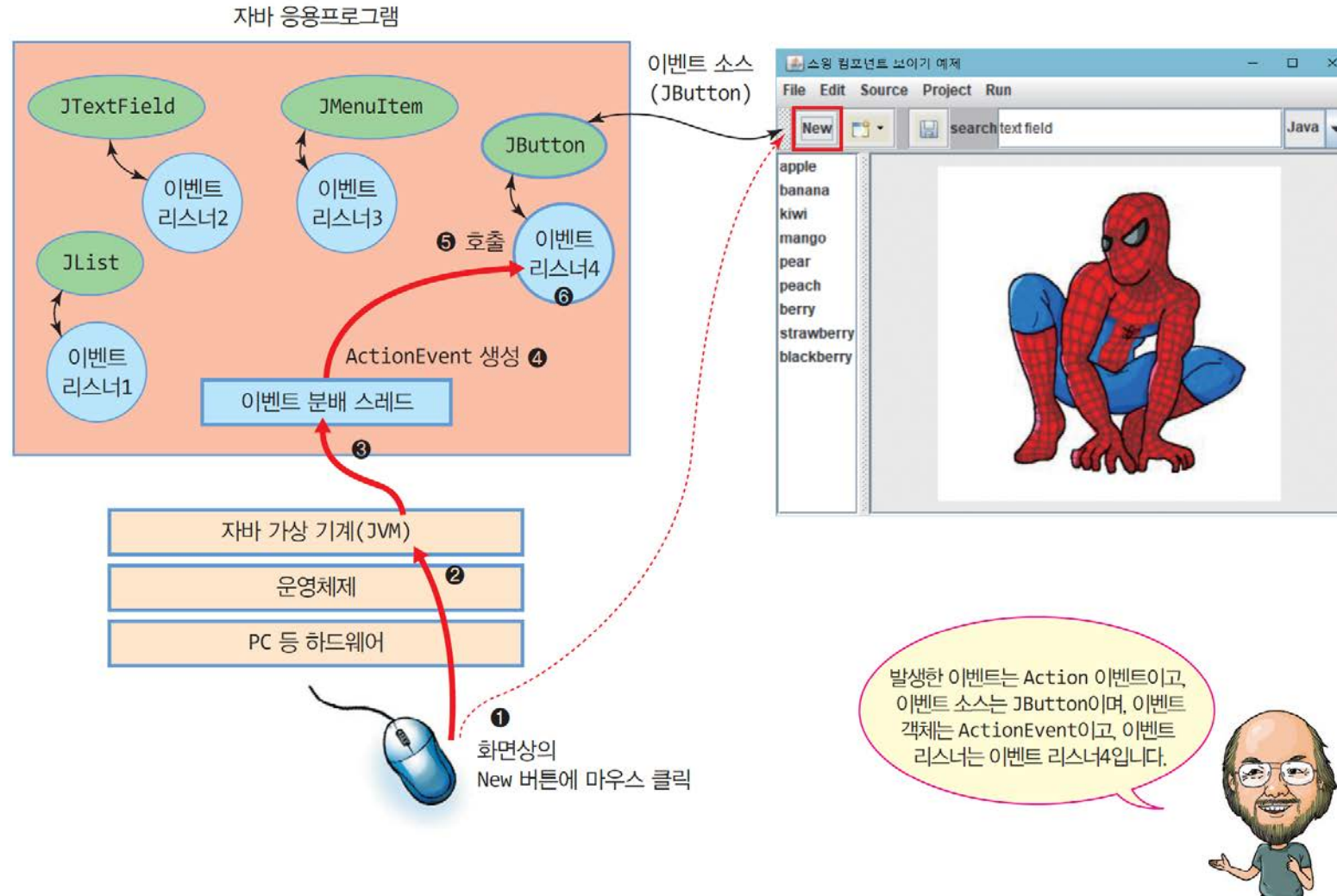
이벤트의 실제 예

3



자바의 이벤트 기반 GUI 응용프로그램 구성

4



이벤트 관련 용어

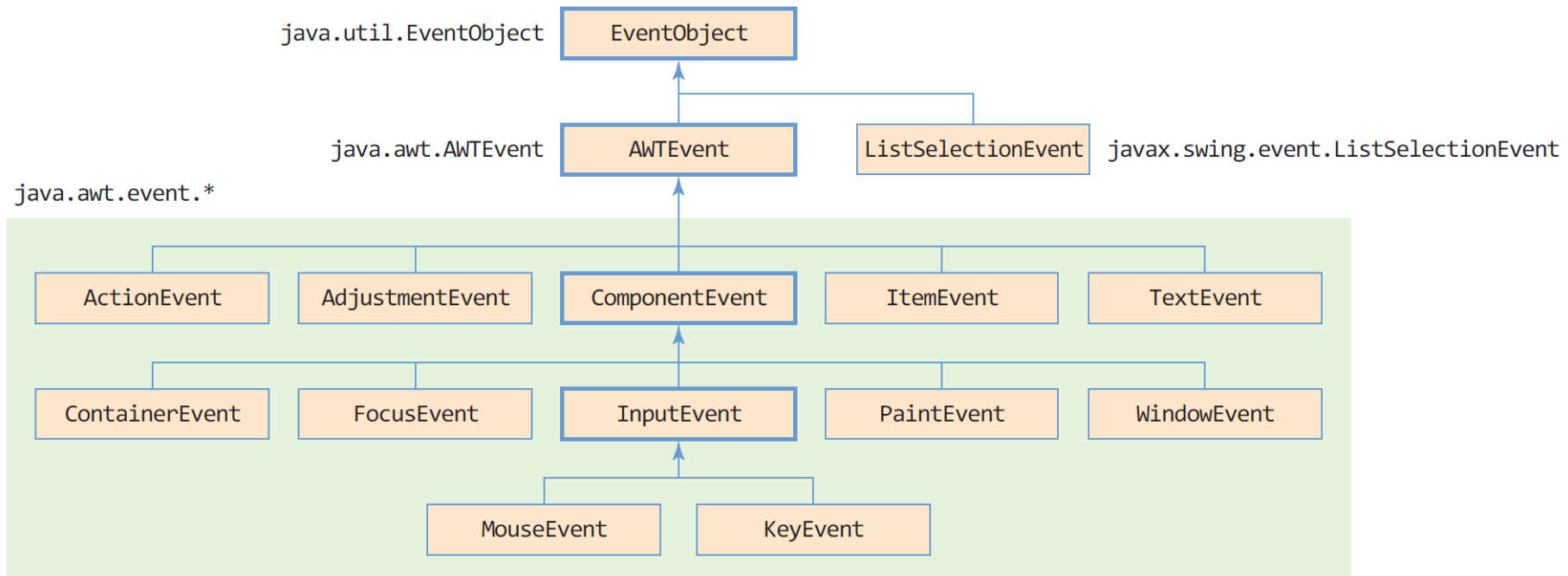
5

- 이벤트 소스
 - ▣ 이벤트를 발생시킨 GUI 컴포넌트
- 이벤트 객체
 - ▣ 발생한 이벤트에 대한 정보
 - 예) 이벤트 종류, 이벤트 소스, 화면 좌표, 마우스 버튼 종류, 눌려진 키
- 이벤트 리스너
 - ▣ 이벤트를 처리하는 코드
 - ▣ 컴포넌트에 등록되어야 작동 가능
- 이벤트 분배 스레드
 - ▣ 동작
 - 자바 가상 기계로부터 이벤트의 발생을 통지 받음
 - 이벤트 소스와 이벤트 종류 결정
 - 적절한 이벤트 객체 생성, 이벤트 리스너를 찾아 호출
 - ▣ 무한 루프를 실행하는 스레드

이벤트 객체

6

- 이벤트 객체란?
 - 이벤트가 발생할 때, 발생한 이벤트에 관한 정보를 가진 객체
 - 이벤트 리스너에 전달됨
 - 이벤트 리스너 코드에서 이벤트가 발생한 상황을 파악할 수 있게 함
- 이벤트 객체의 종류



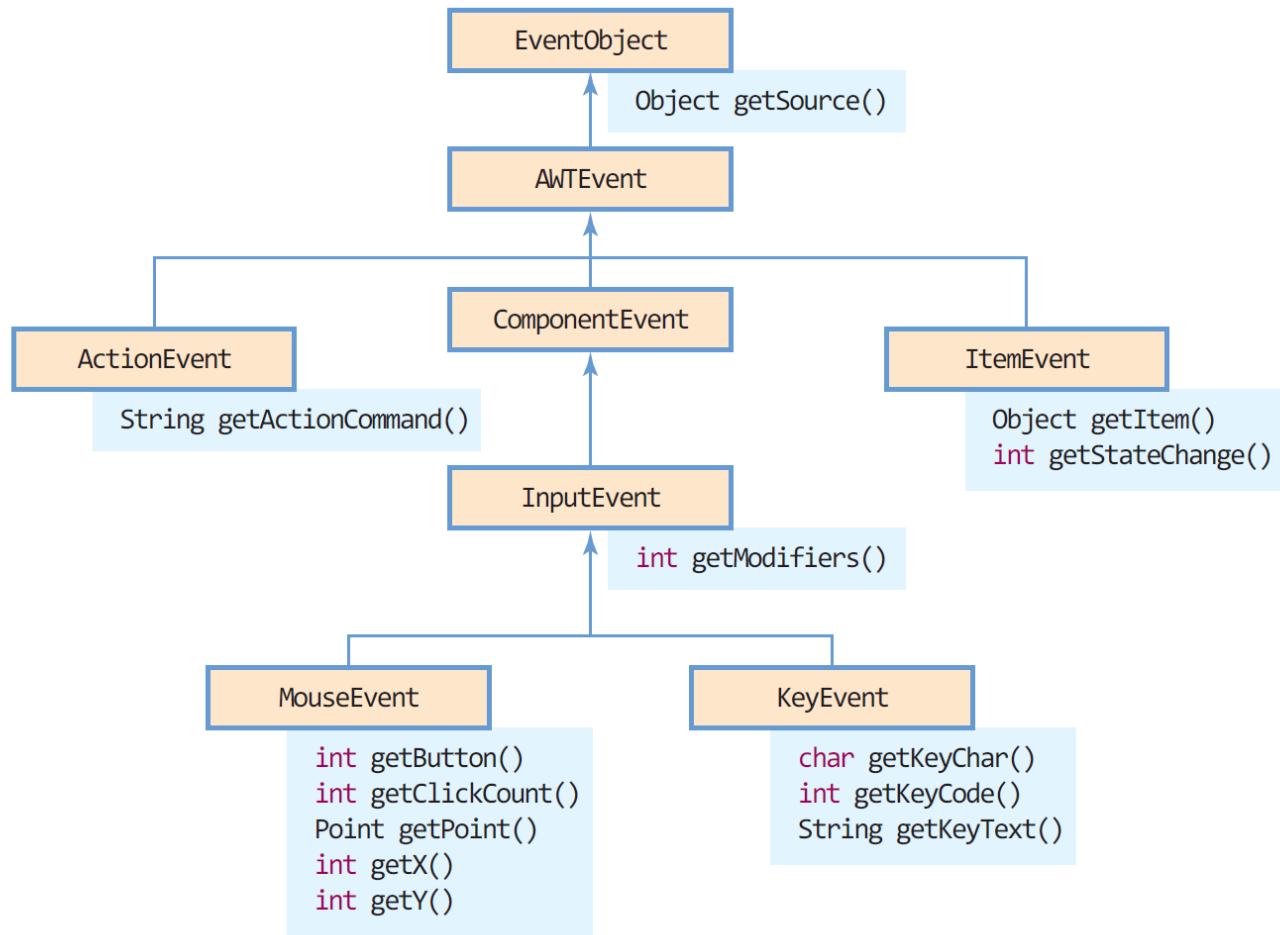
이벤트 객체에 포함된 정보

7

- 이벤트 객체가 포함하는 정보
 - ▣ 이벤트 종류
 - ▣ 이벤트 소스
 - ▣ 이벤트가 발생한 화면 좌표
 - ▣ 이벤트가 발생한 컴포넌트 내 좌표
 - ▣ 버튼이나 메뉴 아이템에 이벤트가 발생한 경우 버튼이나 메뉴 아이템의 문자열
 - ▣ 클릭된 마우스 버튼 번호
 - ▣ 마우스의 클릭 횟수
 - ▣ 키가 눌리었다면 키의 코드 값과 문자 값
 - ▣ 체크박스, 라디오버튼 등과 같은 컴포넌트에 이벤트가 발생하였다면 체크 상태
- 이벤트에 따라 조금씩 다른 정보 포함
 - `ActionEvent` 객체 : 액션 문자열
 - `MouseEvent` 객체 : 마우스의 위치 정보, 마우스 버튼, 함께 눌려진 키 정보 등
 - `ItemEvent` 객체 : 아이템의 체크 상태
- 이벤트 소스 알아 내기
 - ▣ `Object EventObject.getSource()`
 - 발생한 이벤트의 소스 컴포넌트 리턴
 - `Object` 타입으로 리턴하므로 캐스팅하여 사용
 - 모든 이벤트 객체에 대해 적용

이벤트 객체의 메소드

8



이벤트 객체와 이벤트 소스

이벤트 객체	이벤트 소스	이벤트가 발생하는 경우
ActionEvent	JButton	마우스나 <Enter> 키로 버튼 선택
	JMenuItem	메뉴 아이템 선택
	TextField	텍스트 입력 중 <Enter> 키 입력
ItemEvent	JCheckBox	체크박스의 선택 혹은 해제
	JRadioButton	라디오버튼의 선택 상태가 변할 때
	JCheckBoxMenuItem	체크박스 메뉴 아이템의 선택 혹은 해제
ListSelectionEvent	JList	리스트에서 선택된 아이템이 변경될 때
KeyEvent	Component	키가 눌러지거나 눌러진 키가 떼어질 때
MouseEvent	Component	마우스 버튼이 눌러지거나 떼어질 때, 마우스 버튼이 클릭될 때, 컴포넌트 위에 마우스가 올라갈 때, 올라간 마우스가 내려올 때, 마우스가 드래그될 때, 마우스가 단순히 움직일 때
FocusEvent	Component	컴포넌트가 포커스를 받거나 잃을 때
WindowEvent	Window	Window를 상속받는 모든 컴포넌트에 대해 윈도우 활성화, 비활성화, 아이콘화, 아이콘에서 복구, 윈도우 열기, 윈도우 닫기, 윈도우 종료
AdjustmentEvent	JScrollBar	스크롤바를 움직일 때
ComponentEvent	Component	컴포넌트가 사라지거나, 나타나거나, 이동, 크기 변경 시
ContainerEvent	Container	Container에 컴포넌트의 추가 혹은 삭제

이벤트 리스너(Event Listener)

10

- 이벤트 리스너란?
 - ▣ 이벤트를 처리하는 코드, 클래스로 작성
- JDK에서 이벤트 리스너 작성을 위한 인터페이스(interface) 제공
 - ▣ 개발자가 리스너 인터페이스의 추상 메소드 구현
 - 이벤트가 발생하면 자바 플랫폼은 리스너 인터페이스의 추상 메소드 호출
 - ▣ 예) ActionListener 인터페이스

```
interface ActionListener { // 아래 메소드를 개발자가 구현해야 함
    public void actionPerformed(ActionEvent e); // Action 이벤트 발생시 호출됨
}
```

- ▣ 예) MouseListener 인터페이스

```
interface MouseListener { // 아래의 5개 메소드를 개발자가 구현해야 함
    public void mousePressed(MouseEvent e); // 마우스 버튼이 눌러지는 순간 호출
    public void mouseReleased(MouseEvent e); // 눌려진 마우스 버튼이 떼어지는 순간 호출
    public void mouseClicked(MouseEvent e); // 마우스가 클릭되는 순간 호출
    public void mouseEntered(MouseEvent e); // 마우스가 컴포넌트 위에 올라가는 순간 호출
    public void mouseExited(MouseEvent e); // 마우스가 컴포넌트 위에서 내려오는 순간 호출
}
```

리스너 인터페이스와 메소드

이벤트 종류	리스너 인터페이스	리스너의 추상 메소드	메소드가 호출되는 경우
Action	ActionListener	void actionPerformed(ActionEvent)	Action 이벤트가 발생하는 경우
Item	ItemListener	void itemStateChanged(ItemEvent)	Item 이벤트가 발생하는 경우
Key	KeyListener	void keyPressed(KeyEvent)	모든 키에 대해 키가 눌러질 때
		void keyReleased(KeyEvent)	모든 키에 대해 눌러진 키가 떼어질 때
		void keyTyped(KeyEvent)	유니코드 키가 입력될 때
Mouse	MouseListener	void mousePressed(MouseEvent)	마우스 버튼이 눌러질 때
		void mouseReleased(MouseEvent)	눌러진 마우스 버튼이 떼어질 때
		void mouseClicked(MouseEvent)	마우스 버튼이 클릭될 때
		void mouseEntered(MouseEvent)	마우스가 컴포넌트 위에 올라올 때
		void mouseExited(MouseEvent)	컴포넌트 위에 올라온 마우스가 컴포넌트를 벗어날 때
Mouse	MouseMotionListener	void mouseDragged(MouseEvent)	마우스를 컴포넌트 위에서 드래그할 때
		void mouseMoved(MouseEvent)	마우스가 컴포넌트 위에서 움직일 때
Focus	FocusListener	void focusGained(FocusEvent)	컴포넌트가 포커스를 받을 때
		void focusLost(FocusEvent)	컴포넌트가 포커스를 잃을 때
ListSelection	ListSelectionListener	void valueChanged(ListSelectionEvent)	JList에 선택된 아이템이 변경될 때
Window	WindowListener	void windowOpened(WindowEvent)	윈도우가 생성되어 처음으로 보이게 될 때
		void windowClosing(WindowEvent)	윈도우의 시스템 메뉴에서 윈도우 닫기를 시도할 때
		void windowIconified(WindowEvent)	윈도우가 아이콘화 될 때
		void windowDeiconfied(WindowEvent)	아이콘 상태에서 원래 상태로 복귀할 때
		void windowClosed(WindowEvent)	윈도우가 닫혔을 때
		void windowActivated(WindowEvent)	윈도우가 활성화될 때
		void windowDeactivated(WindowEvent)	윈도우가 비활성화될 때
Adjustment	AdjustmentListener	void adjustmentValueChanged(AdjustmentEvent)	스크롤바를 움직일 때
Component	ComponentListener	void componentHidden(ComponentEvent)	컴포넌트가 보이지 않는 상태로 될 때
		void componentShown(ComponentEvent)	컴포넌트가 보이는 상태로 될 때
		void componentResized(ComponentEvent)	컴포넌트의 크기가 변경될 때
		void componentMoved(ComponentEvent)	컴포넌트의 위치가 변경될 때
Container	ContainerListener	void componentAdded(ContainerEvent)	컴포넌트가 컨테이너에 추가될 때
		void componentRemoved(ContainerEvent)	컴포넌트가 컨테이너에서 삭제될 때

Tip : 리스너 등록 메소드가 addXXXListener인 이유?

12

- 컴포넌트는 다른 이벤트에 대한 리스너를 동시에 가질 수 있다.
 - JButton.addActionListener(); // Action 리스너
 - JButton.addKeyListener(); // Key 리스너
 - JButton.addFocusListener(); // Focus 리스너
- 컴포넌트는 한 이벤트에 대해 여러 개의 리스너를 동시에 가질 수 있다.
 - JButton.addActionListener(new MyButtonListener1());
 - JButton.addActionListener(new MyButtonListener2());
 - JButton.addActionListener(new MyButtonListener3());
 - 이때, 리스너는 등록된 반대 순으로 모두 실행된다.

이벤트 리스너 작성 방법

13

□ 3 가지 방법

1. 독립 클래스로 작성

- 이벤트 리스너를 완전한 클래스로 작성
- 이벤트 리스너를 여러 곳에서 사용할 때 적합

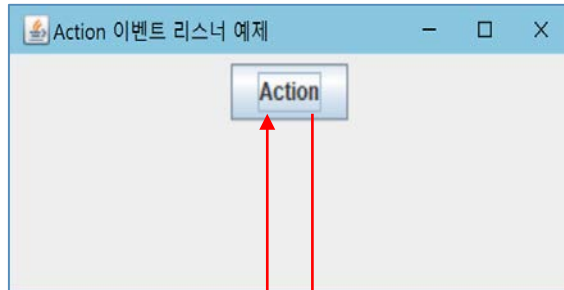
2. 내부 클래스(inner class)로 작성

- 클래스 안에 멤버처럼 클래스 작성
- 이벤트 리스너를 특정 클래스에서만 사용할 때 적합

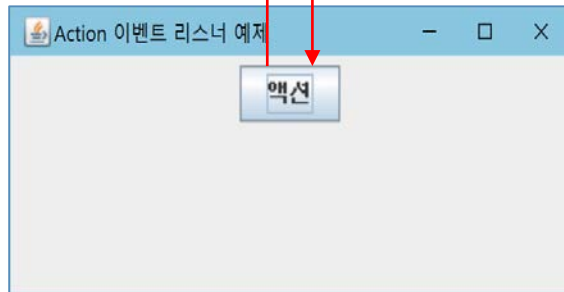
3. 익명 클래스(anonymous class)로 작성

- 클래스의 이름 없이 간단히 리스너 작성
- 클래스 조차 만들 필요 없이 리스너 코드가 간단한 경우에 적합

예제 10-1 : 독립 클래스로 Action 이벤트의 리스너 작성



버튼 클릭



MyActionListener.java
파일로 작성하여도 됨

```
import javax.swing.*;
import java.awt.event.*;
import java.awt.*;
```

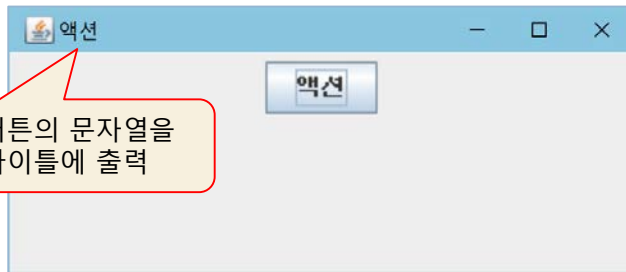
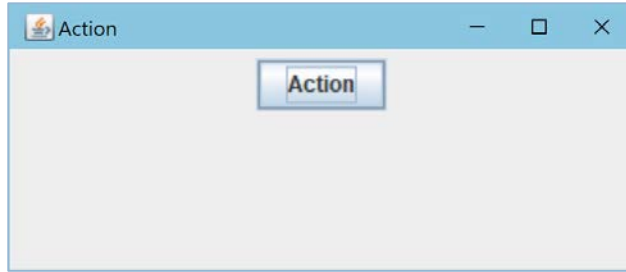
```
public class IndepClassListener extends JFrame {
    public IndepClassListener() {
        setTitle("Action 이벤트 리스너 예제");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        Container c = getContentPane();
        c.setLayout(new FlowLayout());
        JButton btn = new JButton("Action");
        btn.addActionListener(new MyActionListener()); // Action 리스너 달기
        c.add(btn);

        setSize(350, 150);
        setVisible(true);
    }

    public static void main(String [] args) {
        new IndepClassListener();
    }
}

class MyActionListener implements ActionListener {
    public void actionPerformed(ActionEvent e) {
        JButton b = (JButton)e.getSource();
        if(b.getText().equals("Action"))
            b.setText("액션");
        else
            b.setText("Action");
    }
}
```

예제 10-2 : 내부 클래스로 Action 이벤트 리스너 만들기



- Action 리스너를 내부 클래스로 작성
- private으로 선언하여 InnerClassListener 외부에서 사용할 수 없게 함
- 리스너에서 InnerClassListener의 멤버에 대한 접근 용이

```
import javax.swing.*;
import java.awt.event.*;
import java.awt.*;
```

```
public class InnerClassListener extends JFrame {
    public InnerClassListener() {
        setTitle("Action 이벤트 리스너 예제");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        Container c = getContentPane();
        c.setLayout(new FlowLayout());
        JButton btn = new JButton("Action");
        btn.addActionListener(new MyActionListener());
        c.add(btn);

        setSize(350, 150);
        setVisible(true);
    }
}
```

```
private class MyActionListener implements ActionListener {
    public void actionPerformed(ActionEvent e) {
        JButton b = (JButton)e.getSource();
        if(b.getText().equals("Action"))
            b.setText("액션");
        else
            b.setText("Action");
        // InnerClassListener의 멤버나 JFrame의 멤버를 호출할 수 있음
        InnerClassListener.this.setTitle(b.getText()); // 프레임 타이틀에
        버튼 문자열을 출력한다.
    }
}
```

```
public static void main(String [] args) {
    new InnerClassListener();
}
```

익명 클래스로 이벤트 리스너 작성

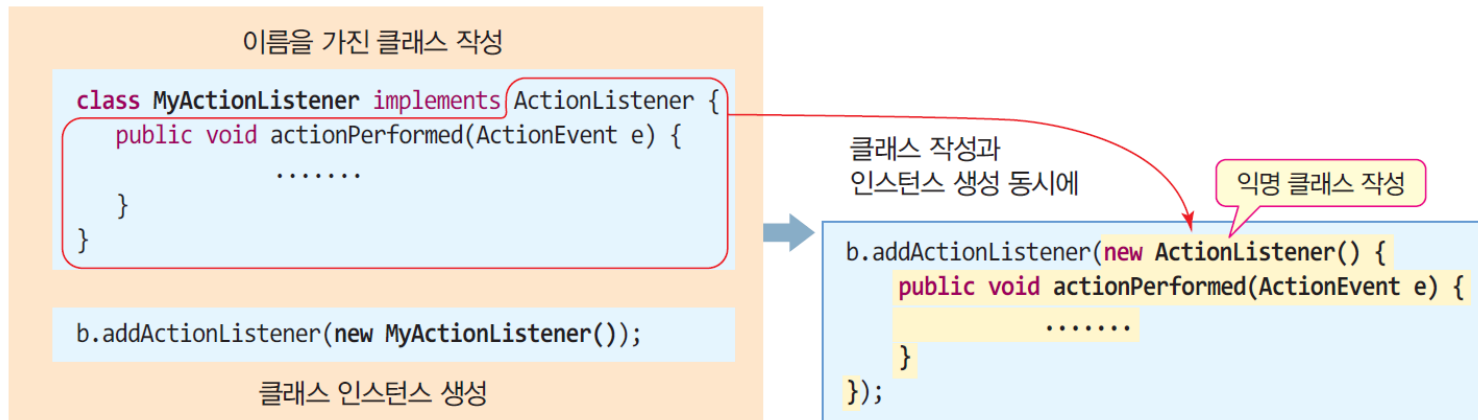
16

□ 익명 클래스란?

- ▣ (클래스 정의 + 인스턴스 생성)을 한번에 작성

```
new 익명클래스의수퍼클래스이름(생성자의 인자들) {  
    .....  
    멤버 구현  
    .....  
};
```

- ▣ ActionListener를 구현하는 익명의 이벤트 리스너 작성 예



(a) 이름을 가진 클래스를 작성하고 클래스 인스턴스 생성하는 경우

(b) ActionListener를 상속받고 바로 메소드 작성, 동시에 new로 인스턴스를 생성하는 경우

예제 10-3 : 익명 클래스로 Action 이벤트 리스너 만들기

익명 클래스로 Action 리스너 작성

AnonymousClassListener의
멤버나 JFrame의 멤버를 호출할 수 있음

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class AnonymousClassListener extends JFrame {
    public AnonymousClassListener() {
        setTitle("Action 이벤트 리스너 작성");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        Container c = getContentPane();
        c.setLayout(new FlowLayout());
        JButton btn = new JButton("Action");
        c.add(btn);

        btn.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                JButton b = (JButton)e.getSource();
                if(b.getText().equals("Action"))
                    b.setText("액션");
                else
                    b.setText("Action");
                setTitle(b.getText());
            }
        });

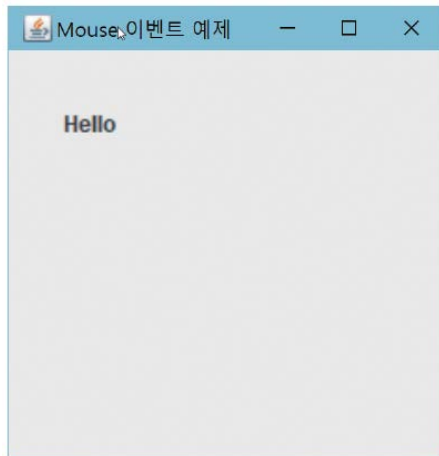
        setSize(350, 150);
        setVisible(true);
    }

    public static void main(String [] args) {
        new AnonymousClassListener();
    }
}
```

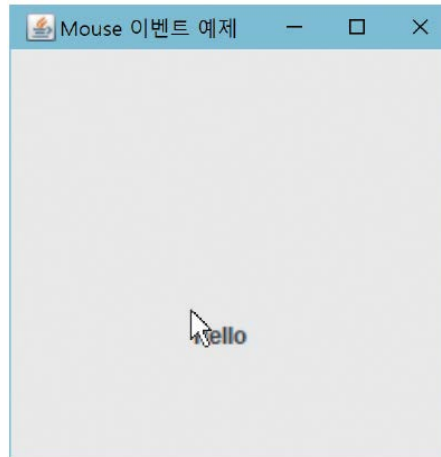
예제 10-4 : 마우스로 문자열 이동시키기 – 마우스 이벤트 연습

18

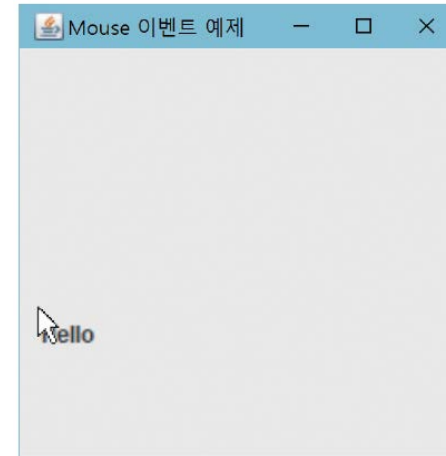
컨텐츠팬의 아무 위치에 마우스 버튼을 누르면 마우스 포인트가 있는 위치로 "hello" 문자열을 옮기는 스윙 응용프로그램을 작성하라.



초기화면



마우스 다른 곳에 클릭한 경우



마우스 다른 곳에 클릭한 경우

예제 10-4의 정답

19

```
import javax.swing.*;
import java.awt.event.*;
import java.awt.*;

public class MouseListenerEx extends JFrame {
    private JLabel la = new JLabel("Hello");

    public MouseListenerEx() {
        setTitle("Mouse 이벤트 예제");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        Container c = getContentPane();
        c.addMouseListener(new MyMouseListener());

        c.setLayout(null);
        la.setSize(50, 20);
        la.setLocation(30, 30);
        c.add(la);

        setSize(250, 250);
        setVisible(true);
    }
}
```

마우스 버튼이 눌러진
위치를 알아내어
la("hello" 문자열)를 그
위치로 옮긴다.

```
class MyMouseListener implements MouseListener {
    public void mousePressed(MouseEvent e) {
        int x = e.getX();
        int y = e.getY();
        la.setLocation(x, y);
    }

    public void mouseReleased(MouseEvent e) {}
    public void mouseClicked(MouseEvent e) {}
    public void mouseEntered(MouseEvent e) {}
    public void mouseExited(MouseEvent e) {}
}

public static void main(String [] args) {
    new MouseListenerEx();
}
```

어댑터(Adapter) 클래스

20

- 이벤트 리스너 구현에 따른 부담 해소를 위해
 - ▣ 리스너 작성시 추상 메소드들을 모두 구현해야 하는 부담
 - 마우스 리스너에서 마우스가 눌러지는 경우(mousePressed())만 처리하고자 하는 경우에도 나머지 4 개의 메소드를 모두 구현해야 하는 부담
- 어댑터 클래스
 - ▣ 리스너의 모든 메소드가 단순 리턴하도록 구현해 놓은 클래스
 - MouseAdapter 예
- ▣ 추상 메소드가 하나뿐인 리스너는 어댑터 클래스 없음
 - ActionListener, ItemAdapter 클래스는 존재하지 않음

```
class MouseAdapter implements MouseListener, MouseMotionListener,  
                                MouseWheelListener {  
    public void mousePressed(MouseEvent e) { }  
    public void mouseReleased(MouseEvent e) { }  
    public void mouseClicked(MouseEvent e) { }  
    public void mouseEntered(MouseEvent e) { }  
    public void mouseExited(MouseEvent e) { }  
    public void mouseDragged(MouseEvent e) { }  
    public void mouseMoved(MouseEvent e) { }  
    public void mouseWheelMoved(MouseWheelEvent e) { }  
}
```

JDK에서 제공하는 어댑터 클래스

21

리스너 인터페이스	대응하는 어댑터 클래스
ActionListener	없음
ItemListener	없음
KeyListener	KeyAdapter
MouseListener	MouseAdapter
MouseMotionListener	MouseMotionAdapter 혹은 MouseAdapter
FocusListener	FocusAdapter
WindowListener	WindowAdapter
AdjustmentListener	없음
ComponentListener	ComponentAdapter
ContainerListener	ContainerAdapter

어댑터 사용 예(MouseAdapter)

22

```
JLabel la;  
contentPane.addMouseListener(new MyMouseListener());  
.....
```

```
class MyMouseListener implements MouseListener {  
    public void mousePressed(MouseEvent e) {  
        int x = e.getX();  
        int y = e.getY();  
        la.setLocation(x, y);  
    }  
    public void mouseReleased(MouseEvent e) {}  
    public void mouseClicked(MouseEvent e) {}  
    public void mouseEntered(MouseEvent e) {}  
    public void mouseExited(MouseEvent e) {}  
}
```

MouseListener를 이용한 경우

```
JLabel la;  
contentPane.addMouseListener(new MyMouseAdapter());  
.....
```

```
class MyMouseAdapter extends MouseAdapter {  
    public void mousePressed(MouseEvent e) {  
        int x = e.getX();  
        int y = e.getY();  
        la.setLocation(x, y);  
    }  
}
```

MouseAdapter를 이용한 경우



예제 10-5 : MouseAdapter 사용하기

23

MouseAdapter를 이용하여 예제 10-4를 수정하라.

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class MouseAdapterEx extends JFrame {
    private JLabel la = new JLabel("Hello");

    public MouseAdapterEx() {
        setTitle("Mouse 이벤트 예제");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        Container c = getContentPane();
        c.addMouseListener(new MyMouseAdapter());

        c.setLayout(null);
        la.setSize(50, 20);
        la.setLocation(30, 30);
        c.add(la);

        setSize(250, 250);
        setVisible(true);
    }
}
```

```
class MyMouseAdapter extends MouseAdapter {
    public void mousePressed(MouseEvent e) {
        int x = e.getX();
        int y = e.getY();
        la.setLocation(x, y);
    }
}

public static void main(String [] args) {
    new MouseAdapterEx();
}
```

Key 이벤트와 포커스

24

- 키 입력 시, 다음 세 경우에 Key 이벤트 발생
 - ▣ 키를 누르는 순간
 - ▣ 누른 키를 떼는 순간
 - ▣ 누른 키를 떼는 순간(Unicode 키의 경우에만)
- 키 이벤트를 받을 수 있는 조건
 - ▣ 모든 컴포넌트 가능하지만, 현재 포커스(focus)를 가진 컴포넌트
- 포커스(focus)
 - ▣ 컴포넌트나 응용프로그램이 키 이벤트를 독점하는 권한
 - ▣ 컴포넌트에 포커스 설정 방법 : 다음 2 라인의 코드 필요

```
component.setFocusable(true); // component가 포커스를 받을 수 있도록 설정  
component.requestFocus(); // componen에 포커스 강제 지정
```

- 자바플랫폼마다 실행 환경의 초기화가 서로 다를 수 있기 때문에 다음 코드가 필요함 `component.setFocusable(true);`

컴포넌트에 포커스 주기

25

- 스윙 프레임이 만들어질 포커스를 주고자 하는 경우
 - ▣ JFrame의 setVisible(true) 이후에 포커스를 주어야 함

```
setVisible(true); // 스윙 프레임 출력
component.setFocusable(true);
component.requestFocus();
```

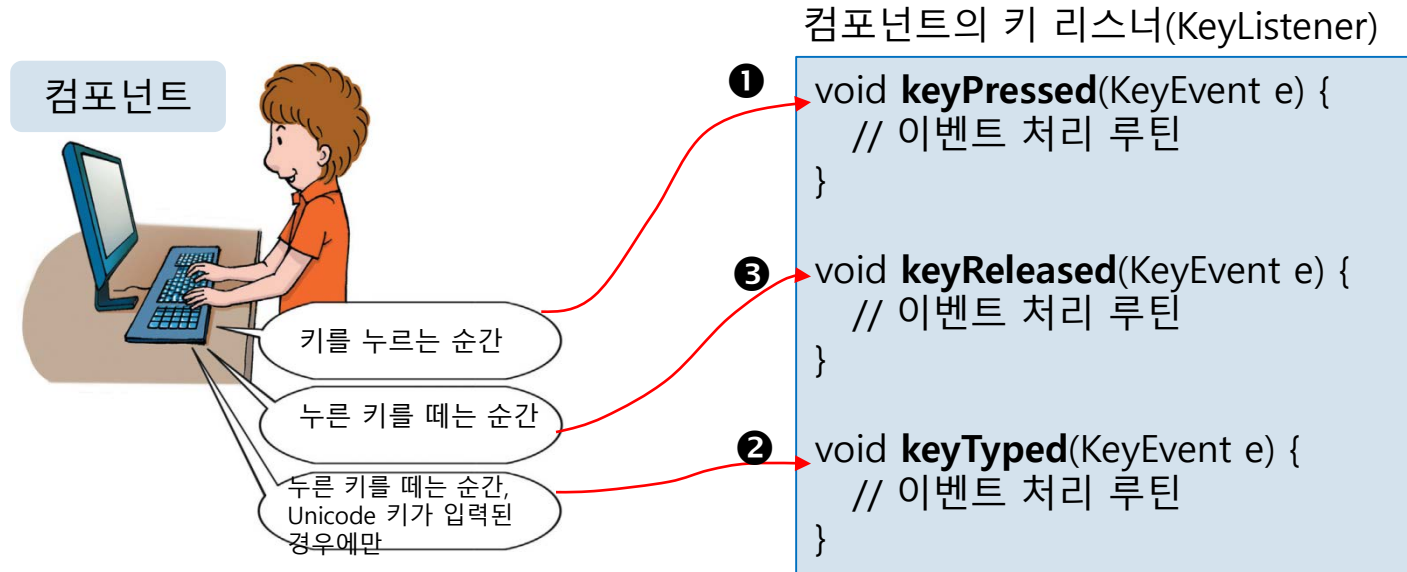
- 마우스로 컴포넌트를 클릭할 때 포커스 지정하는 방법
 - ▣ 언제든지 필요할 때 포커스 줄 수 있음

```
component.addMouseListener(new MouseAdapter() {
    public void mouseClicked(MouseEvent e) {
        Component c = (Component)e.getSource(); // 클릭된 컴포넌트
        c.setFocusable(true);
        c.requestFocus();
    }
}); // 예제 10-8에서 활용하였음
```

KeyListener의 메소드와 키

26

KeyListener의 3 개 메소드



KeyListener의 메소드가 실행되는 순서 ① ② ③

컴포넌트에 키 이벤트 리스너 등록

```
component.addKeyListener(myKeyListener);
```

키는 2가지 종류

27

1. 유니코드 키

- 유니코드는 전 세계의 **문자**에 대한 코드 체계
- 문자들에 대해서만 유니 코드 값 정의
 - A~Z, a~z, 0~9, !, @, & 등
- 유니코드 키가 눌러진 경우 이벤트 호출 순서
 - keyPressed(), keyTyped(), keyReleased() 순으로 호출

2. 유니코드가 아닌 키

- 문자 키가 아닌 다음 키들(제어 키)
 - <Function>, <Home>, <Up>, <Delete>, <Control>, <Shift>, <Alt> 등
- 정의된 유니코드 값 없음
- 키마다 **키 코드 값(가상 키 코드 값)**이 정의되어 있음
- 유니코드 키가 아닌 경우 키 이벤트 호출 순서
 - keyPressed(), keyReleased() 만 호출됨

□ 가상 키

- 유니코드 키든 아니든 모든 키에 자바의 가상 키 코드가 정의되어 있음
 - 가상 키 값으로 어떤 키인지 비교 판단 가능

가상 키(Virtual Key)

28

- 가상 키 코드는 KeyEvent 클래스에 상수로 선언

가상 키	설명	가상 키	설명
VK_0 ~ VK_9	0에서 9까지의 키, '0'~'9'까지의 유니코드 값과 동일	VK_LEFT	왼쪽 방향 키
VK_A ~ VK_Z	A에서 Z까지의 키, 'A'~'Z'까지의 유니코드 값과 동일	VK_RIGHT	오른쪽 방향 키
VK_F1 ~ VK_F24	<F1>~<F24>까지의 키 코드	VK_UP	<Up> 키
VK_HOME	<Home> 키	VK_DOWN	<Down> 키
VK_END	<End> 키	VK_CONTROL	<Control> 키
VK_PGUP	<Page Up> 키	VK_SHIFT	<Shift> 키
VK_PGDN	<Page Down> 키	VK_ALT	<Alt> 키
VK_UNDEFINED	입력된 키의 코드 값을 알 수 없음	VK_TAB	<Tab> 키

입력된 키 판별

29

- 키가 입력되면 키 정보를 가진 이벤트 객체 생성 : KeyEvent 객체
 - KeyEvent 객체가 리스너에 전달됨
- 1. 키의 **문자 코드**(유니코드) 알아내기, `char KeyEvent.getKeyChar()`
 - ▣ 눌려진 키에 해당하는 문자 코드(유니코드) 리턴
 - ▣ 눌려진 키가 문자 키인 경우에만 작동
- 2. 입력된 키의 **가상 키** 값 알아내기, `int KeyEvent.getKeyCode()`
 - ▣ 모든 키에 대해 작동
 - ▣ 입력된 키를 판별하기 위해 가상키(Virtual Key) 값과 비교
 - 가상 키 값은 KeyEvent 클래스의 상수로 정의됨
- 3. **키 이름** 문자열 리턴 `String KeyEvent.getKeyText(int keyCode)`
 - ▣ Static 메소드
 - ▣ 매개변수 keyCode의 코드 값(가상 키)에 해당하는 키의 이름 문자열 리턴
 - F1 키의 경우 "F1", Shift 키의 경우 "SHIFT" 등의 문자열 리턴

KeyEvent의 getKeyChar()과 getKeyCode()

30

	a 키를 누르는 순간	<pre>public void keyPressed(KeyEvent e) { char keyChar = e.getKeyChar(); int keyCode = e.getKeyCode(); }</pre>	keyChar	키 a의 유니코드 값('a')	keyCode	VK_A
	<F5> 키를 누르는 순간	<pre>public void keyPressed(KeyEvent e) { char keyChar = e.getKeyChar(); int keyCode = e.getKeyCode(); }</pre>	keyChar	CHAR_UNDEFINED	keyCode	VK_F5
	w 키를 떼는 순간	<pre>public void keyTyped(KeyEvent e) { char keyChar = e.getKeyChar(); int keyCode = e.getKeyCode(); }</pre>	keyChar	키 w의 유니코드 값('w')	keyCode	VK_UNDEFINED
	<F5> 키를 떼는 순간	<pre>public void keyTyped(KeyEvent e) { char keyChar = e.getKeyChar(); int keyCode = e.getKeyCode(); }</pre>	keyChar		keyCode	

<F5> 키에 대해서는
keyTyped() 메소드가
호출되지 않습니다.



예제 10-6 : 다양한 KeyEvent와 KeyListener 활용

31

```
import javax.swing.*;
import java.awt.event.*;
import java.awt.*;

public class KeyListenerEx extends JFrame {
    private JLabel [] keyMessage;

    public KeyListenerEx() {
        setTitle("keyListener 예제");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        Container c = getContentPane();
        c.setLayout(new FlowLayout());

        c.addKeyListener(new MyKeyListener());

        keyMessage = new JLabel [3];
        keyMessage[0] = new JLabel(" getKeyCode() ");
        keyMessage[1] = new JLabel(" getKeyChar() ");
        keyMessage[2] = new JLabel(" getKeyText() ");

        for(int i=0; i<keyMessage.length; i++) {
            c.add(keyMessage[i]);
            keyMessage[i].setOpaque(true);
            keyMessage[i].setBackground(Color.YELLOW);
        }
    }
}
```

```
setSize(300,150);
setVisible(true);

c.setFocusable(true);
c.requestFocus();
}

class MyKeyListener extends KeyAdapter {
    public void keyPressed(KeyEvent e) {
        int keyCode = e.getKeyCode();
        char keyChar = e.getKeyChar();

        keyMessage[0].setText(Integer.toString(keyCode));
        keyMessage[1].setText(Character.toString(keyChar));
        keyMessage[2].setText(e.getKeyText(keyCode));
    }
}

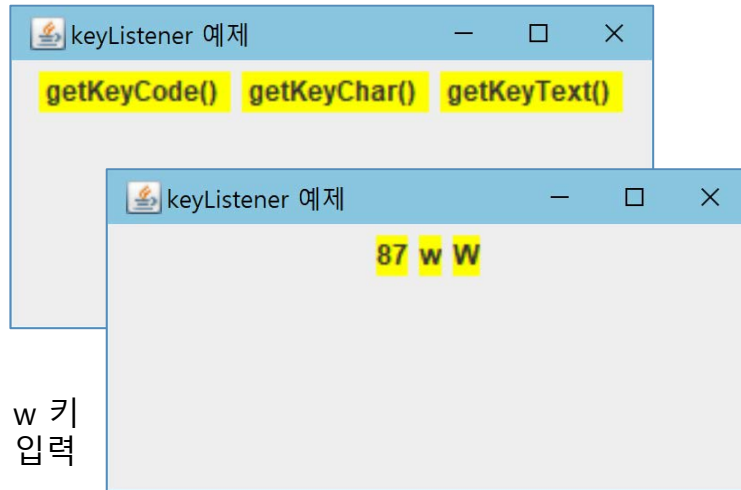
public static void main(String [] args) {
    new KeyListenerEx();
}
```

컴포넌트의 바탕색이 보이도록 하기 위해서는
컴포넌트가 불투명하기 지정될 필요 있음

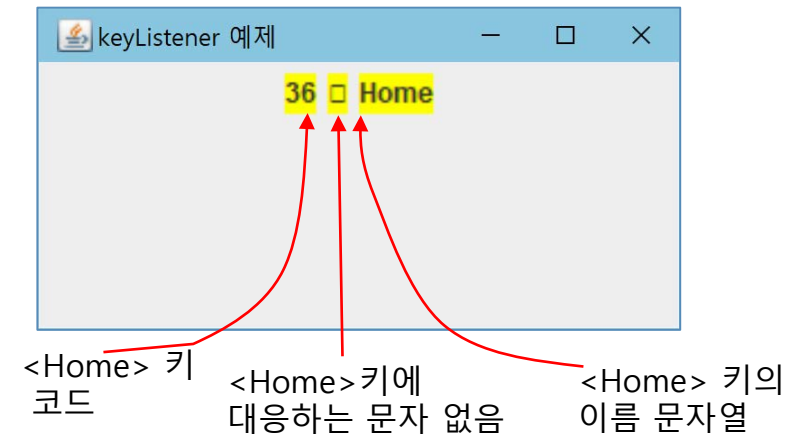
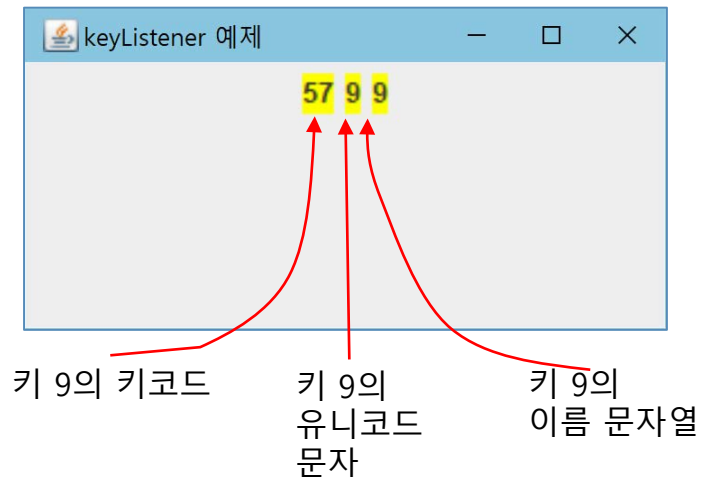
실행 결과

32

초기화면



<Control> 키 입력



예제 10-7 : <F1> 키를 입력받으면 콘텐츠팬의 배경을 초록색으로, % 키를 입력받으면 노란색으로 변경

33

```
import javax.swing.*;
import java.awt.event.*;
import java.awt.*;

public class KeyCodeEx extends JFrame {
    private JLabel la = new JLabel();

    public KeyCodeEx() {
        setTitle("Key Code 예제 : F1키:초록색, % 키 노란색");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        Container c = getContentPane();

        c.addKeyListener(new MyKeyListener());
        c.add(la);

        setSize(300,150);
        setVisible(true);

        c.setFocusable(true);
        c.requestFocus();
    }
}
```

키 입력을 받을 수 있도록 포커스를 준다.

```
class MyKeyListener extends KeyAdapter {
    public void keyPressed(KeyEvent e) {
        la.setText(e.getKeyText(e.getKeyCode()));

        if(e.getKeyChar() == '%')
            contentPane.setBackground(Color.YELLOW);
        else if(e.getKeyCode() == KeyEvent.VK_F1)
            contentPane.setBackground(Color.GREEN);
    }
}

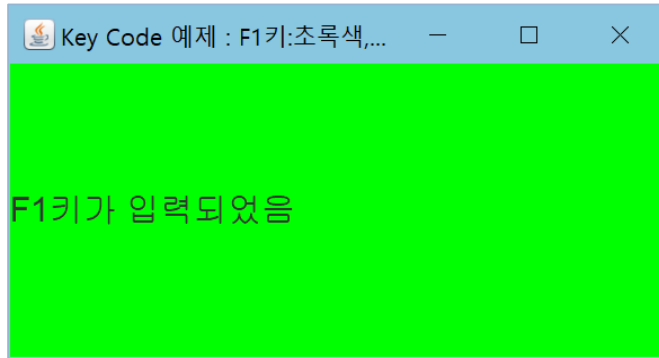
public static void main(String [] args) {
    new KeyCodeEx();
}
```

% 키를 판별하기 위해 e.getKeyChar() 호출

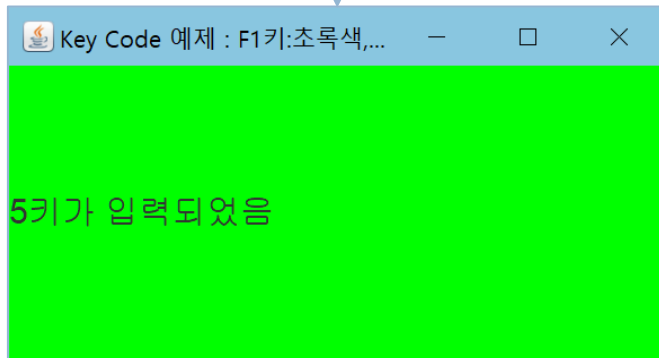
F1 키를 판별하기 위해 e.getKeyCode() 호출
KeyEvent.VK_F1 값과 비교

예제 10-7 실행

34

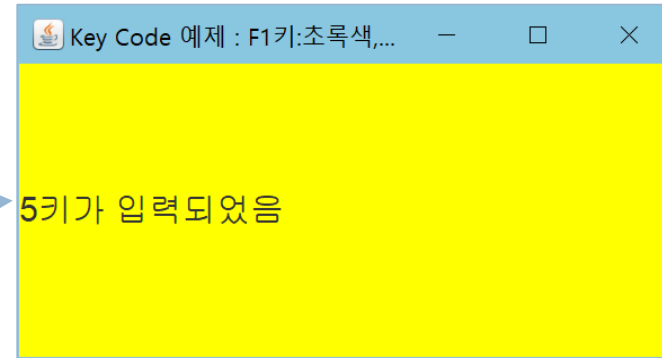


<F1> 키를 누른 경우



키 5 를 누르면 노란색 배경으로 변하지 않음.

%키나 5키는 키보드에서 동일한 키이지만
if(e.getKeyChar() == '%')로 비교하였기 때문



% 키를 누른 경우 노란색으로 변경

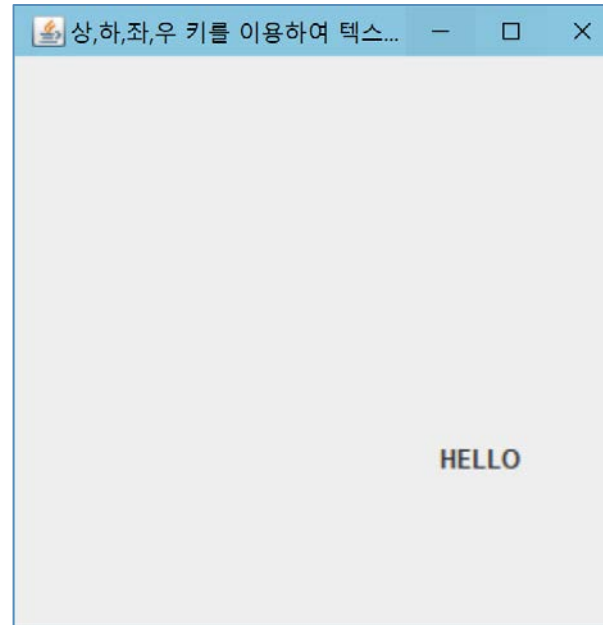
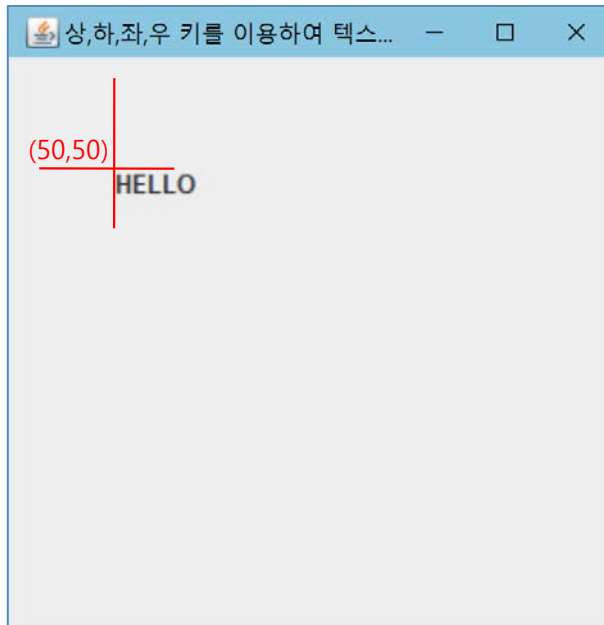
% 키는 Shift키+5키이므로,
키 5에 대한 문자열 출력

예제 10-8 : 상(UP), 하(DOWN), 좌(LEFT), 우(RIGHT) 키로 "HELLO" 문자열을 마음대로 움직이기

35

상, 하, 좌, 우 키를 이용하여 "HELLO" 문자열을 움직이는 응용프로그램을 작성하라.

"HELLO" 문자열은 JLabel 컴포넌트로 만들어 컨테이너에 부착하고 상, 하, 좌, 우 키를 움직이면 키 방향으로 한 번에 10픽셀씩 움직인다. 이를 위해 컨테이너의 배치관리자를 삭제하여야 한다. "HELLO" 문자열을 초기에 (50, 50) 위치에 출력하라.



상,하,좌,우 키를 움직이면 한 번에 10픽셀씩 "HELLO" 텍스트는 상,하,좌,우로 이동한다. 이 텍스트는 프레임의 영역을 벗어나서 움직일 수 있다.

예제 소스: 상,하,좌,우 키로 텍스트 움직이기

36

```
import javax.swing.*;
import java.awt.event.*;
import java.awt.*;

public class FlyingTextEx extends JFrame {
    private final int FLYING_UNIT = 10;
    private JLabel la = new JLabel("HELLO");

    public FlyingTextEx() {
        setTitle("상,하,좌,우 키를 이용하여 텍스트 움직이기");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        Container c = getContentPane();
        c.setLayout(null);
        c.addKeyListener(new MyKeyListener());

        la.setLocation(50,50);
        la.setSize(100,20);
        c.add(la);
        setSize(300,300);
        setVisible(true);
        c.setFocusable(true);
        c.requestFocus();

        c.addMouseListener(new MouseAdapter() {
            public void mouseClicked(MouseEvent e) {
                Component com = (Component)e.getSource();
                com.setFocusable(true);
                com.requestFocus();
            }
        });
    }
}
```

```
class MyKeyListener extends KeyAdapter {
    public void keyPressed(KeyEvent e) {
        int keyCode = e.getKeyCode();

        switch(keyCode) {
            case KeyEvent.VK_UP:
                la.setLocation(la.getX(), la.getY()-FLYING_UNIT);
                break;
            case KeyEvent.VK_DOWN:
                la.setLocation(la.getX(), la.getY()+FLYING_UNIT);
                break;
            case KeyEvent.VK_LEFT:
                la.setLocation(la.getX()-FLYING_UNIT, la.getY());
                break;
            case KeyEvent.VK_RIGHT:
                la.setLocation(la.getX()+FLYING_UNIT, la.getY());
                break;
        }
    }
}

public static void main(String [] args) {
    new FlyingTextEx();
}
```

마우스 이벤트와 마우스 관련 리스너

37

□ 마우스 이벤트

▣ 사용자의 마우스 조작에 따라 발생하는 이벤트, 8가지 경우

Mouse 이벤트가 발생하는 경우	리스너의 메소드	리스너
마우스가 컴포넌트 위에 올라갈 때	<code>void mouseEntered(MouseEvent e)</code>	<code>MouseListener</code>
마우스가 컴포넌트에서 내려올 때	<code>void mouseExited(MouseEvent e)</code>	<code>MouseListener</code>
마우스 버튼이 눌러졌을 때	<code>void mousePressed(MouseEvent e)</code>	<code>MouseListener</code>
눌러진 버튼이 떼어질 때	<code>void mouseReleased(MouseEvent e)</code>	<code>MouseListener</code>
마우스로 컴포넌트를 클릭하였을 때	<code>void mouseClicked(MouseEvent e)</code>	<code>MouseListener</code>
마우스가 드래그되는 동안	<code>void mouseDragged(MouseEvent e)</code>	<code>MouseMotionListener</code>
마우스가 움직이는 동안	<code>void mouseMoved(MouseEvent e)</code>	<code>MouseMotionListener</code>
마우스 휠이 구르는 동안	<code>void mouseWheelMoved (MouseWheelEvent e)</code>	<code>MouseWheelListener</code>

▣ 마우스가 눌러진 위치에서 떼어지는 경우 메소드 호출 순서

`mousePressed()`, `mouseReleased()`, `mouseClicked()`

▣ 마우스가 드래그될 때 호출되는 메소드 호출 순서

`mousePressed()`, `mouseDragged()`, `mouseDragged()`, ..., `mouseDragged()`, `mouseReleased()`

마우스 리너스 달기

38

- MouseListener의 5 개의 이벤트를 처리하는 경우
 - mouseEntered(), mouseExited(), mousePressed(), mouseReleased(), mouseClicked()
 - ▣ 마우스 리스너 등록
 - component.**addMouseListener**(myMouseListener);

- MouseMotionListener의 이벤트도 함께 처리하고자 하는 경우
 - mouseDragged(), mouseMoved()
 - ▣ 마우스 모션 리스너 등록 필요
 - component.**addMouseMotionListener**(myMouseMotionListener);

MouseEvent 객체로부터 얻을 수 있는 정보

39

- 마우스 포인터의 위치
 - ▣ int getX(), int getY(),
 - ▣ Point getPoint()
- 입력된 마우스 버튼
 - ▣ int getButton()
- 마우스 클릭 횟수
 - ▣ int getClickCount()

```
public void mousePressed(MouseEvent e) {  
    int x = e.getX();  
    int y = e.getY();  
}
```

```
public void mousePressed(MouseEvent e) {  
    if(e.getButton() == MouseEvent.BUTTON1)  
        System.out.println("Left Button Pressed");  
}
```

```
public void mouseClicked(MouseEvent e) {  
    if(e.getClickCount() == 2) {  
        // 더블클릭을 처리하는 루틴  
    }  
}
```

예제 10-9 : 마우스와 마우스 모션 이벤트 활용

40

```
import javax.swing.*;
import java.awt.event.*;
import java.awt.*;

public class MouseListenerAllEx extends JFrame {
    private JLabel la = new JLabel("No Mouse Event");

    public MouseListenerAllEx() {
        setTitle("MouseListener와 MouseMotionListener 예제");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        Container c = getContentPane();
        c.setLayout(new FlowLayout());

        MyMouseListener listener =
            new MyMouseListener();
        c.addMouseListener(listener);
        c.addMouseMotionListener(listener);

        c.add(la);

        setSize(300,200);
        setVisible(true);
    }
}
```

```
class MyMouseListener implements MouseListener,
    MouseMotionListener {

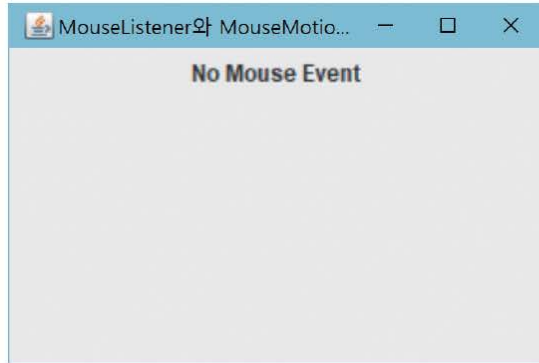
    public void mousePressed(MouseEvent e) {
        la.setText("mousePressed (" + e.getX() + ", " + e.getY() + ")");
    }
    public void mouseReleased(MouseEvent e) {
        la.setText("MouseReleased (" + e.getX() + ", " + e.getY() + ")");
    }
    public void mouseClicked(MouseEvent e) {}
    public void mouseEntered(MouseEvent e) {
        Component c = (Component)e.getSource();
        c.setBackground(Color.CYAN);
    }
    public void mouseExited(MouseEvent e) {
        Component c = (Component)e.getSource();
        c.setBackground(Color.YELLOW);
    }

    public void mouseDragged(MouseEvent e) {
        la.setText("MouseDragged (" + e.getX() + ", " + e.getY() + ")");
    }
    public void mouseMoved(MouseEvent e) {
        la.setText("MouseMoved (" + e.getX() + ", " + e.getY() + ")");
    }
}

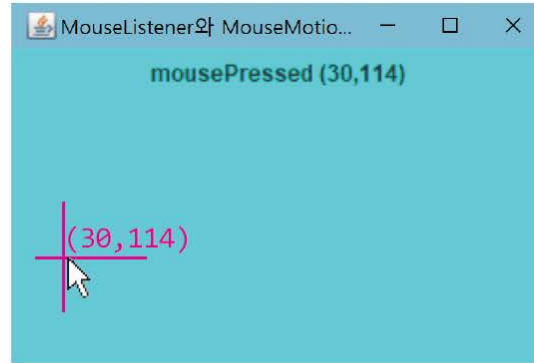
public static void main(String [] args) {
    new MouseListenerAllEx();
}
}
```


예제 10-9 실행

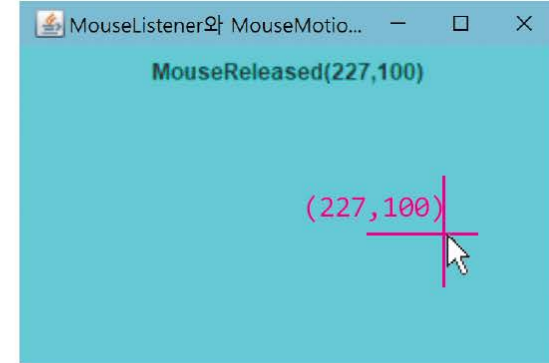
41



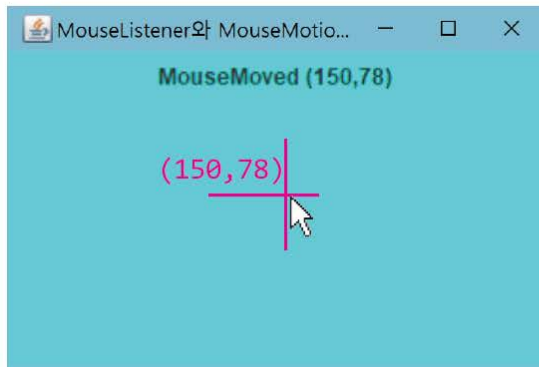
초기 화면



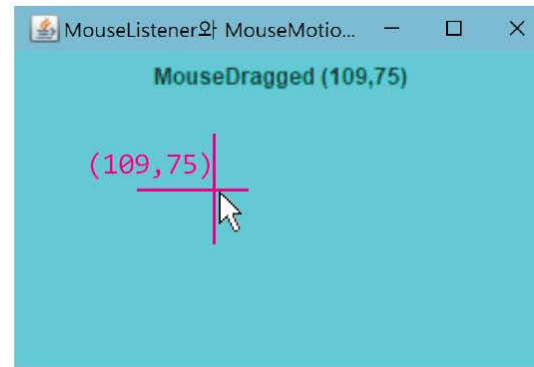
mouseEntered()에 의해 배경색 변경.
마우스 버튼이 눌러진 순간



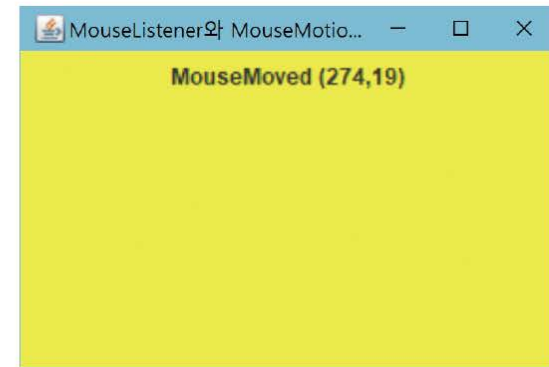
눌러진 마우스 버튼이 떼어진 순간



마우스가 패널 위에 이동하는 동안



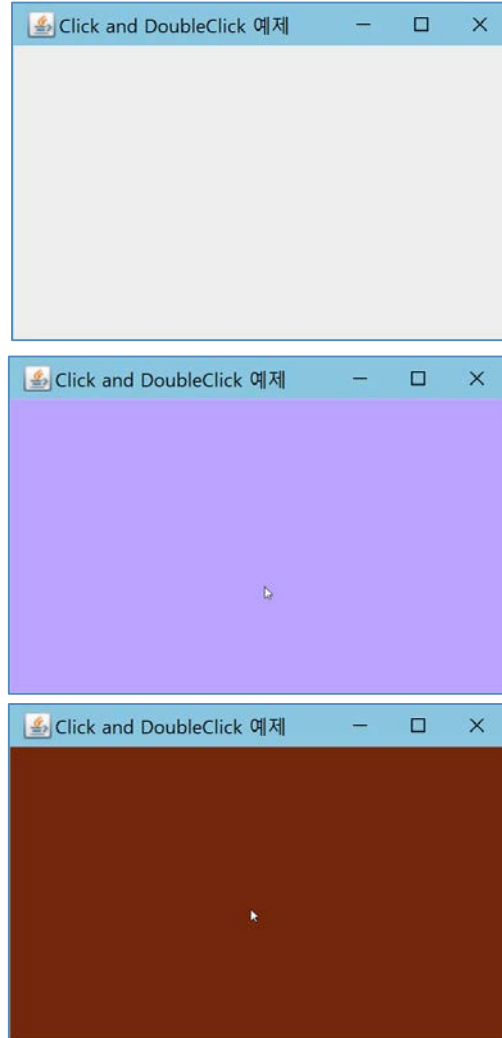
마우스가 패널 위에 드래그하는 동안



마우스가 패널 바깥으로 나가면
mouseExited()에 의해 배경색 변경

예제 10-10 : 더블클릭 시 콘텐츠팬의 배경색 변경

더블클릭할 때마다 콘텐츠팬의 배경색을 랜덤하게 변경한다.



```
import javax.swing.*;
import java.awt.event.*;
import java.awt.*;

public class ClickAndDoubleClickEx extends JFrame {
    public ClickAndDoubleClickEx() {
        setTitle("Click and DoubleClick 예제");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        Container c = getContentPane();
        c.addMouseListener(new MyMouseListener());
        setSize(300,200);
        setVisible(true);
    }

    class MyMouseListener extends MouseAdapter {
        public void mouseClicked(MouseEvent e) {
            if(e.getClickCount() == 2) {
                int r = (int)(Math.random()*256);
                int g = (int)(Math.random()*256);
                int b = (int)(Math.random()*256);
                Component c = (Component)e.getSource();
                c.setBackground(new Color(r,b,g));
            }
        }
    }

    public static void main(String [] args) {
        new ClickAndDoubleClickEx();
    }
}
```

MouseEvent와 MouseWheelListener

43

- 마우스 휠 이벤트
 - ▣ 마우스 휠이 구를 때마다 발생하는 이벤트
 - 이벤트 발생시마다 MouseEvent 객체 생성
- MouseWheelListener
 - ▣ 마우스 휠 이벤트 리스너
 - ▣ 마우스 휠 이벤트가 발생할 때마다 호출되는 메소드
 - Public void mouseWheelMoved(MouseEvent e)
- 마우스 휠 리스너 구현

```
component.addMouseWheelListener(new MouseWheelListener() {  
    public void mouseWheelMoved(MouseEvent e) {  
        // 마우스 휠의 구르는 방향에 따라 이벤트를 처리한다.  
    }  
});
```

- 실습문제 10-7 참고